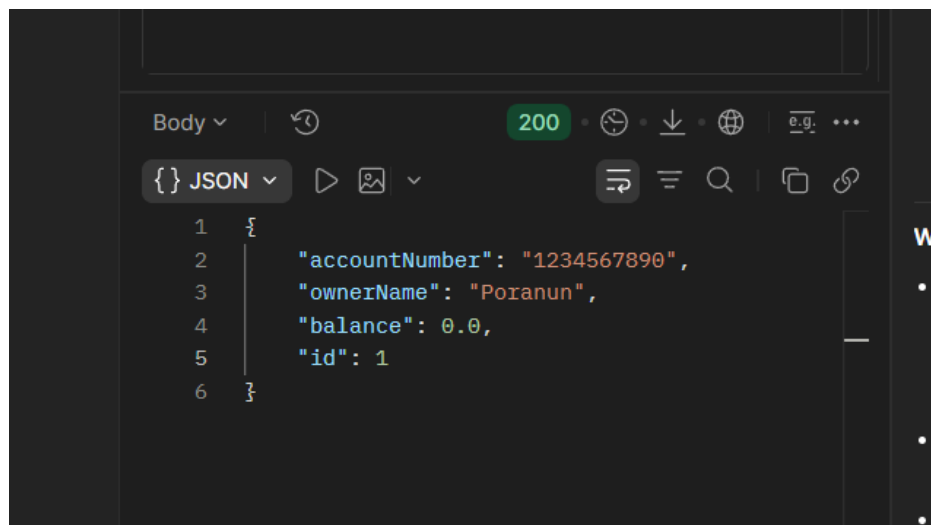


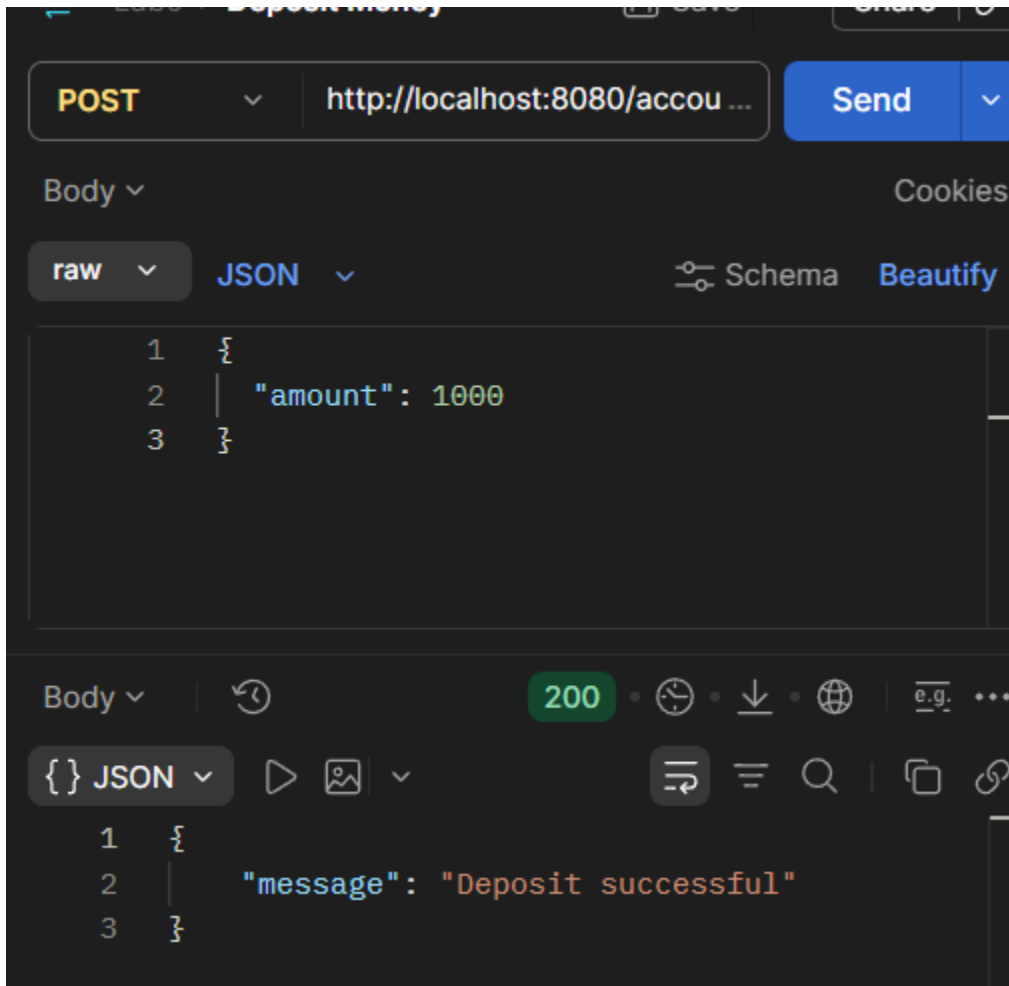
Lab08:Table Relationships

สร้าง Account



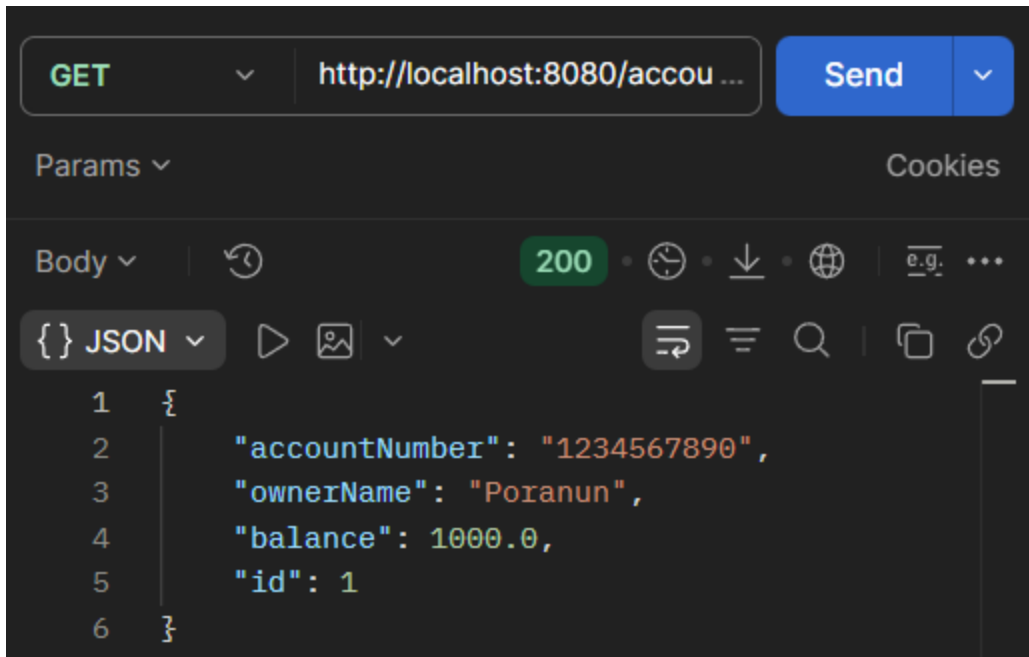
อธิบายผล : ภาพนี้แสดงผลการทดลองสร้างบัญชีธนาคาร (Create Account) ผ่านโปรแกรม Postman โดยระบบสามารถสร้างบัญชีได้สำเร็จ และส่งผลลัพธ์กลับมาเป็นข้อมูลในรูปแบบ JSON

การฝากเงินสำเร็จ



อธิบายผล: จากการทดลองฝากเงินจำนวน 1,000 บาทเข้าสู่บัญชีที่มี id เท่ากับ 1 ระบบสามารถดำเนินการฝากเงินได้สำเร็จ โดย Response แสดงข้อความ "Deposit successful" และได้รับ Status 200 OK แสดงว่าการส่ง Request สำเร็จตามที่กำหนดไว้ในระบบ

ตรวจสอบ Account



อธิบายผล: หลังจากทำการฝากเงินจำนวน 1,000 บาท ระบบสามารถเรียกดูข้อมูลบัญชีได้สำเร็จ โดยค่า balance เปลี่ยนจาก 0 เป็น 1000.0 แสดงว่าระบบได้เพิ่มยอดเงินในบัญชีเรียบร้อยแล้ว และข้อมูล Account ถูกบันทึกลงฐานข้อมูลอย่างถูกต้อง

ตรวจสอบ DepositTransaction

The screenshot shows a database management interface. On the left, a tree view displays the database structure, including 'lab8shop' and 'lab9'. Under 'lab9', there are several categories like 'Casts', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas (1)', and 'Aggregates'. The 'Schemas (1)' category is expanded, showing a 'public' schema. On the right, the 'Query' tab is active, displaying the SQL query: `SELECT * FROM deposit_transaction;`. Below the query, the 'Data Output' tab shows the results of the query. The results are displayed in a table with 4 columns: 'id' (bigint, PK), 'amount' (double precision), and 'account_id' (bigint). The table shows 1 row of data: id=1, amount=1000, account_id=1. The 'Messages' and 'Notifications' tabs are also visible but empty.

	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1

อธิบายผล: จากการตรวจสอบฐานข้อมูล พบรายการฝากเงินจำนวน 1 รายการ โดยมีจำนวนเงินฝากเท่ากับ 1,000 บาท และมีค่า account_id อ้างอิงไปยังบัญชีที่ทำการฝากเงิน แสดงว่าระบบสามารถบันทึกประวัติการฝากเงินลงในตาราง DepositTransaction ได้สำเร็จ

ทดลอง Rollback

Lab9 > Deposit Money

POST http://localhost:8080/accou ... Send

Body raw JSON Schema Beautify

```
1 {
2   "amount": 1000
3 }
```

Body 500 Investigate server error Diagnose Decline

{ } JSON

```
1 {
2   "timestamp": "2026-09-14T23:37:18.991Z",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/accounts/1/deposit"
6 }
```

Lab9 > Get Account

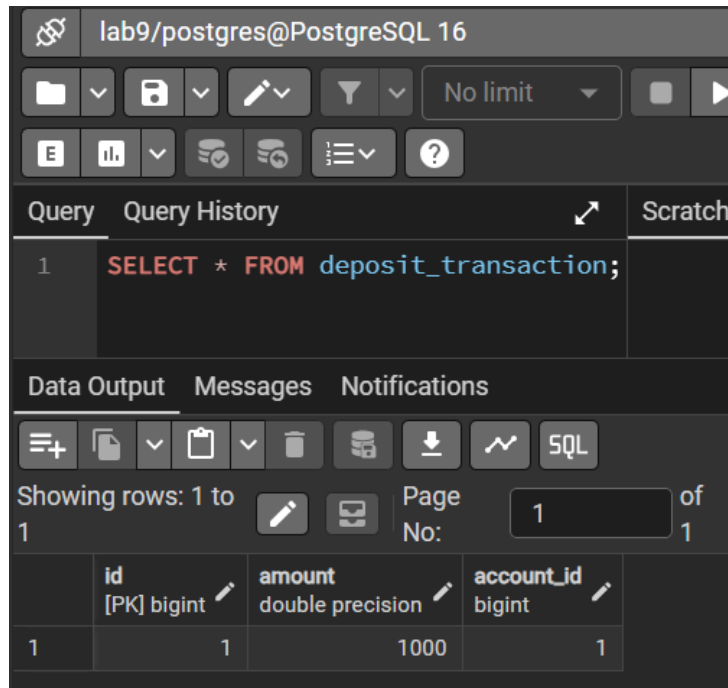
GET http://localhost:8080/accou ... Send

Params Cookies

Body 200

{ } JSON

```
1 {
2   "accountNumber": "1234567890",
3   "ownerName": "Poranun",
4   "balance": 1000.0,
5   "id": 1
6 }
```



อธิบายผล : จากการทดลองได้เพิ่มคำสั่ง throw new RuntimeException("Test Rollback"); หลังจากบันทึก Account และ DepositTransaction เพื่อจำลองการเกิดข้อผิดพลาดในระบบ

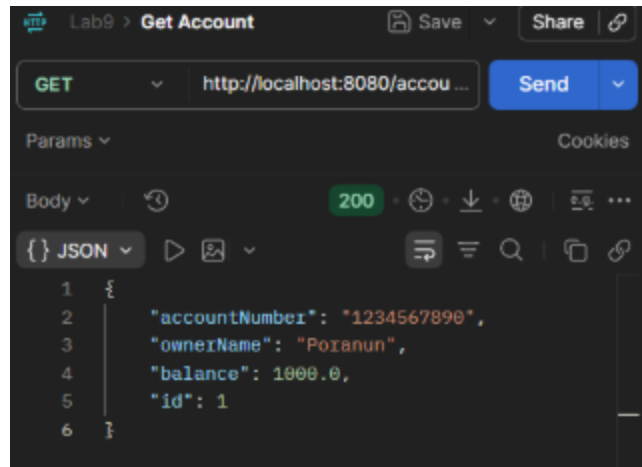
เมื่อส่ง Request ฝากเงินจำนวน 1,000 บาท ระบบตอบกลับด้วย Status 500 Internal Server Error เนื่องจากเกิด RuntimeException

จากนั้นตรวจสอบข้อมูล Account พบว่ายอดเงินยังคงเป็น 1,000 บาทเท่าเดิม และเมื่อตรวจสอบตาราง DepositTransaction พบว่าไม่มีรายการฝากเงินใหม่เพิ่มขึ้น

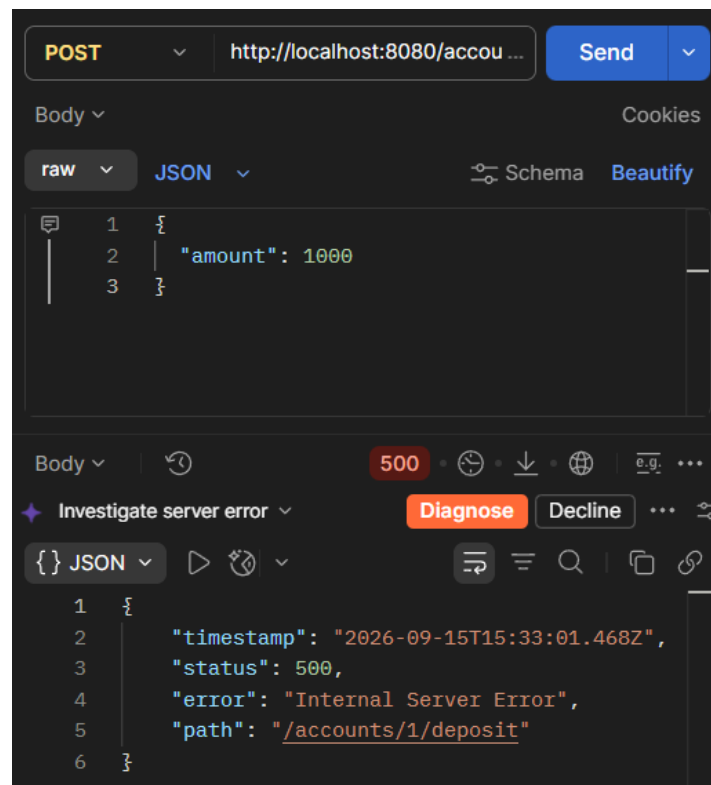
สรุปได้ว่า เมื่อใช้ @Transactional การทำงานของการฝากเงินจะถูกควบคุมให้อยู่ใน Transaction เดียวกัน เมื่อเกิดข้อผิดพลาด ระบบจะทำการ ROLLBACK การเปลี่ยนแปลงกลับไปยังสถานะก่อนเริ่ม Transaction ทำให้ข้อมูล Account และ DepositTransaction ยังคงสอดคล้องกัน

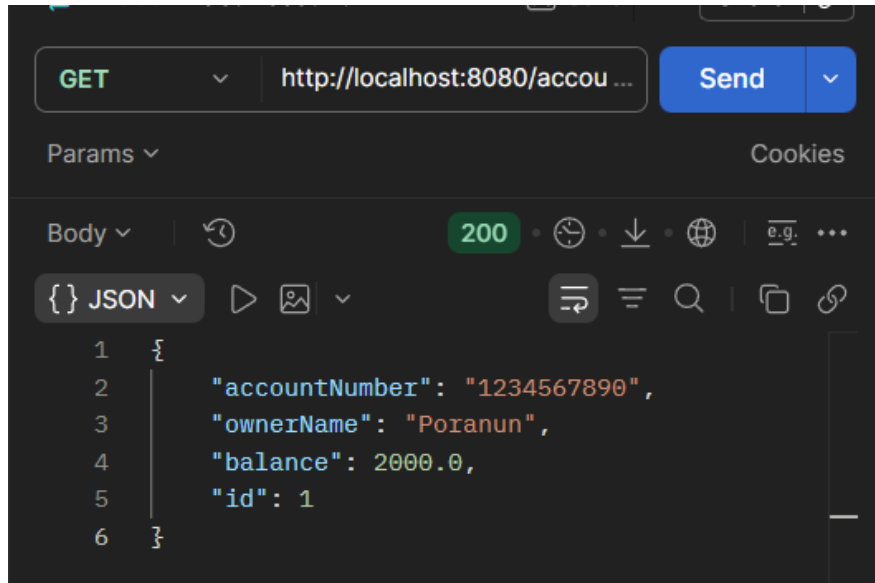
เปรียบเทียบมี @Transactional กับไม่มี @Transactional

1.มี @Transactional



2.ไม่ @Transactional





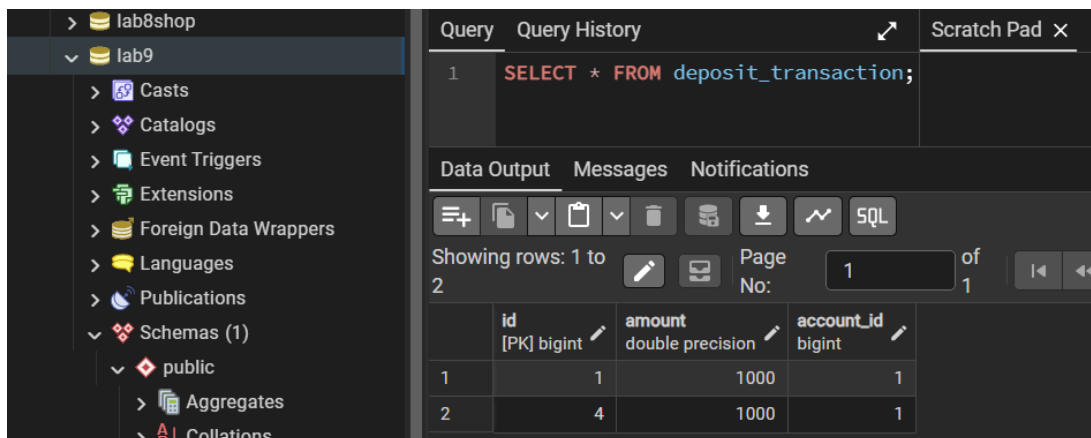
อธิบายผล : จากการเปรียบเทียบการทำงานของระบบฝากเงิน พบว่า @Transactional มีหน้าที่ควบคุมการทำงานของคำสั่งต่าง ๆ ในเมธอดให้เป็น Transaction เดียวกัน

ในกรณีที่มี @Transactional เมื่อเกิด Error หลังจากบันทึกข้อมูล ระบบจะทำการ ROLLBACK การเปลี่ยนแปลงทั้งหมด ทำให้ยอดเงินใน Account และข้อมูล DepositTransaction กลับไปเป็นเหมือนก่อนเริ่มการฝากเงิน

ส่วนกรณีที่ไม่มี @Transactional จะไม่มี Transaction เดียวที่ครอบคลุมการทำงานของทั้งหมดของเมธอด ดังนั้นเมื่อเกิด Error หลังจากมีการบันทึกข้อมูลบางส่วน ข้อมูลที่บันทึกไปแล้วอาจยังคงอยู่ในฐานข้อมูลได้

จากการทดลองสามารถสรุปได้ว่า การใช้ @Transactional ช่วยรักษาความถูกต้องและความสอดคล้องของข้อมูล โดยทำให้การเพิ่มยอดเงินและการบันทึกประวัติฝากเงินสำเร็จหรือถูกยกเลิกไปพร้อมกัน

ภาพจาก database



คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

คำตอบ @Transactional เป็น Annotation ของ Spring ที่ใช้กำหนดขอบเขตการทำงานของ Transaction โดยรวบรวมคำสั่งที่เกี่ยวข้องกันให้อยู่ใน Transaction เดียวกัน หากการทำงานสำเร็จทั้งหมดจะทำการ COMMIT แต่ถ้าเกิดข้อผิดพลาดตามเงื่อนไขการ Rollback ระบบจะย้อนกลับการเปลี่ยนแปลงของ Transaction นั้น

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

คำตอบ เพราะการฝากเงินมีการเปลี่ยนแปลงข้อมูลมากกว่า 1 ตาราง ได้แก่ การเพิ่มยอดเงินใน Account และการบันทึกประวัติการฝากเงินใน DepositTransaction จึงควรใช้ Transaction เพื่อให้ข้อมูลทั้งสองส่วนถูกบันทึกสำเร็จหรือถูกยกเลิกพร้อมกัน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

คำตอบ COMMIT = ยืนยันข้อมูล / ROLLBACK = ยกเลิกข้อมูล

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

คำตอบ Account และ DepositTransaction มีความสัมพันธ์แบบ One-to-Many (1:N)

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

คำตอบ เมื่อเกิด Error ภายใน Transaction และเข้าเงื่อนไขการ Rollback ระบบจะยกเลิกการเปลี่ยนแปลงทั้งหมดที่อยู่ใน Transaction เดียวกัน ทำให้ข้อมูลใน Database กลับไปเป็นสถานะก่อนเริ่ม Transaction